TranslationManager

System Description

Abstract
This document describes the TranslationManager support system of the Eclipse Arrowhead framework (documentation edition 5.3.0), as implemented in `ah5-TranslationManager-java-spring` of the primary Java source tree (`Eclipse-Arrowhead-java`) and deployed with the standalone Compose definition of that folder.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Significant Prior Art 4
2.2 How This System Is Meant to Be Used 4
2.3 System functionalities and properties 4
2.4 Important Delimitations 5

3 Services **6**
3.1 Produced services 6
3.2 Consumed services 6

4 Security **7**
4.1 Security model 7

5 References **7**

6 Revision History **9**
6.1 Amendments 9
6.2 Quality Assurance 9

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the TranslationManager system. TranslationManager enables automatic interface and data-model translation: it finds providers a consumer could use through a translation bridge, builds the bridge with interface and data-model translator systems, and tracks the bridges' use.



Figure 1: SysML-style block view of TranslationManager with its produced and consumed services.

The rest of this document is organized as follows. In Section 2.1, we reference major prior art capabilities of the system. In Section 2.2, we describe the intended usage of the system. In Section 2.3, we describe fundamental properties provided by the system. In Section 2.4, we describe delimitations of capabilities of the system. In Section 3, we describe the abstract service functions consumed or produced by the system. In Section 4, we describe the security capabilities of the system.

2.1 Significant Prior Art

Successor of the Arrowhead 4.x Translator and Gateway concepts (AITIA 5.2.2 implementation).

2.2 How This System Is Meant to Be Used

DynamicServiceOrchestration uses `translationBridgeManagement` during orchestration with `ALLOW_TRANSLATION`; translators report with `translationReport`; consumers may use `translationBridge` directly.

2.3 System functionalities and properties

2.3.1 Functional properties of the system

- Discover translation candidates among service instances.
- Negotiate bridges with interface and data-model translators and issue bridge tokens.
- Record bridge state and usage reports; abort bridges; query bridges.

2.3.2 Configuration of system properties

The system is configured with Spring properties (`application.properties`, overridden by environment variables in the container deployment). The properties that shape its behaviour are:

Property	Default	Meaning
`server.port`	8465	HTTPS port.
`authentication.policy`	declared	Standalone deployment: `outsourced` with Authentication identity tokens and mutual TLS.
`management.policy` / `management.whitelist`	authorization	Access to the management services.
`enable.blacklist.filter` / `force.blacklist.filter`	false / true	Blacklist check of requesters.
`enable.authorization`	true	Check consumer authorization of candidates.
`allow.discovery.flags`	true	Accept discovery flags in management discovery.
`translator.service.min.availability`	5	Minimum remaining availability (minutes) of translator services.
`normalization.mode`	extended	Name normalization.
`mqtt.api.enabled`	false	Offer the MQTT interfaces.
`sysmonitor.enabled`, `sysmonitor.broker.*`, `sysmonitor.ssl.*`	true, amqp-broker:5671, server.ssl stores	Native `sysMonitor` publisher of the common library.
`arrowhead.ssl.crl-file`	(unset)	CRL checked for every client certificate (standalone deployment: `/crl/ca.crl`).

Secrets (database, keystore and identity passwords, secret keys) have no usable defaults; the standalone deployment reads them from per-deployment Compose file secrets under `/run/secrets`.

The standalone deployment of this folder uses `config/application.properties` (copied into the image) with `server.port`=8465, `authentication.policy=outsourced`, `service.interface`.

`policy=CERT_AUTH`, `management.policy=sysop-only`, `service.authorization.enabled=true`, `mqtt.api.enabled=false`, `arrowhead.ssl.crl-file=/crl/ca.crl`; secrets are read from Compose file secrets under `/run/secrets` (`spring.config.import=configtree:/run/secrets/`). `enable.blacklist.filter` is not enabled there, so incoming requests are not checked against the Blacklist; blacklisted providers are still removed by DynamicServiceOrchestration.

2.3.3 Data stored by the system

Bridge headers, discovery and details records, and log entries (database `ah_translation_manager`).

2.3.4 Non functional properties

Discovery and negotiation call ConsumerAuthorization, the ServiceRegistry and the translators synchronously.

2.3.5 Stateful or stateless

Stateful: bridges are persistent.

2.4 Important Delimitations

The TranslationManager does not translate; without translator systems in the cloud no bridge can be built. The standalone deployment starts no translator; translators are separate systems that register their own services.

3 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

3.1 Produced services

Service	Version	Interfaces	SD	IDD
`translationReport`	1.0.0	HTTP(S), MQTT(S)	[1]	[2], [3]
`translationBridge`	1.0.0	HTTP(S), MQTT(S)	[4]	[5], [6]
`translationBridge Management`	1.0.0	HTTP(S), MQTT(S)	[7]	[8], [9]
`generalManagement`	1.0.0	HTTP(S), MQTT(S)	[10]	[11], [12]
`sysMonitor`	1.0.0	AMQP(S)	[13]	[14]

The HTTP interfaces are always offered, the MQTT interfaces with `mqtt.api.enabled=true` and the AMQP `sysMonitor` interface with `sysmonitor.enabled=true`. The unregistered endpoints of the implementation are: `GET /translation/monitor/echo`.

3.2 Consumed services

Provider	Service	Operations	Use	Condition
ServiceRegistry	systemDiscovery, serviceDiscovery	register, revoke, lookup	Registration of the system and its services; lookup of the bootstrap services.	always
Authentication	identity	identity-login, identity-verify	Own identity token; verification of requester tokens.	`authentication.policy=outsourced`
Consumer Authorization	authorization	verify	Management filter decisions.	`management.policy=authorization`
Blacklist	blacklistDiscovery	check	Blacklist filter.	`enable.blacklist.filter=true`
Consumer Authorization	authorizationToken Management	generate-tokens	Bridge tokens.	bridges
Translators	interface and data-model translation services	initialize, abort	Building and closing bridges.	bridges
AMQP broker (SysMonitor)	sysMonitor (publisher)	echo, power, status	Own observations.	`sysmonitor.enabled=true`

Except for the bootstrap services, which are looked up in the ServiceRegistry, the system reaches other systems' services through DynamicServiceOrchestration (`http.collector.mode=ORCHESTRATION`, results cached for 300 s); every such call needs a ConsumerAuthorization grant.

4 Security

The system can run in an insecure mode (`server.ssl.enabled=false`, plain HTTP, identity declared by the requester) for development, and in the Arrowhead secure mode with HTTPS and X.509 certificates following the Arrowhead certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`). Certificates not issued under the cloud CA are refused by the TLS handshake. The standalone deployment of this folder (`config/application.properties` and `compose.yaml`) runs the secure mode.

4.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates; MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; the database connection uses TLS with server verification in the standalone deployment. Secrets are kept out of the container environment (Compose file secrets).
- **System authentication:** by the authentication policy: `certificate`, `outsourced` (identity token verified with Authentication) or `declared` (insecure); the standalone deployment uses `outsourced`. An identity token is accepted only together with the client certificate of the system it was issued to (token-to-certificate binding), and every client certificate is checked against the cloud CA's CRL (fail closed).
- **Authorization of produced services:** management services are protected by the management filter (`management.policy`: `SYSOP_ONLY`, `WHITELIST` or `AUTHORIZATION`); with `service.authorization.enabled=true` ConsumerAuthorization decides each call; an optional blacklist filter refuses blacklisted requesters. Sysop identities bypass the management filter.
- Bridges are created only for authorized candidates (`enable.authorization=true`).

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

5 References

[1] J. Delsing, "translationReport – Service Description (TranslationManager)," Eclipse Arrowhead, SD 5.3.0, 2026, translationManager-v5.3.0-translationReport-SD.pdf.

[2] ——, "translationReport HTTP/TLS/JSON – Interface Design Description (TranslationManager)," Eclipse Arrowhead, IDD 5.3.0, 2026, translationManager-v5.3.0-translationReport-HTTP-IDD.pdf.

[3] ——, "translationReport MQTT/TLS/JSON – Interface Design Description (TranslationManager)," Eclipse Arrowhead, IDD 5.3.0, 2026, translationManager-v5.3.0-translationReport-MQTT-IDD.pdf.

[4] ——, "translationBridge – Service Description (TranslationManager)," Eclipse Arrowhead, SD 5.3.0, 2026, translationManager-v5.3.0-translationBridge-SD.pdf.

[5] ——, "translationBridge HTTP/TLS/JSON – Interface Design Description (TranslationManager)," Eclipse Arrowhead, IDD 5.3.0, 2026, translationManager-v5.3.0-translationBridge-HTTP-IDD.pdf.

[6] ——, "translationBridge MQTT/TLS/JSON – Interface Design Description (TranslationManager)," Eclipse Arrowhead, IDD 5.3.0, 2026, translationManager-v5.3.0-translationBridge-MQTT-IDD.pdf.

[7] ——, "translationBridgeManagement – Service Description (TranslationManager)," Eclipse Arrowhead, SD 5.3.0, 2026, translationManager-v5.3.0-translationBridgeManagement-SD.pdf.

[8] ——, "translationBridgeManagement HTTP/TLS/JSON – Interface Design Description (TranslationManager)," Eclipse Arrowhead, IDD 5.3.0, 2026, translationManager-v5.3.0-translationBridgeManagement-HTTP-IDD.pdf.

[9] ——, “translationBridgeManagement MQTT/TLS/JSON – Interface Design Description (TranslationManager),” Eclipse Arrowhead, IDD 5.3.0, 2026, translationManager-v5.3.0-translationBridgeManagement-MQTT-IDD.pdf.

[10] ——, “generalManagement – Service Description (TranslationManager),” Eclipse Arrowhead, SD 5.3.0, 2026, translationManager-v5.3.0-generalManagement-SD.pdf.

[11] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (TranslationManager),” Eclipse Arrowhead, IDD 5.3.0, 2026, translationManager-v5.3.0-generalManagement-HTTP-IDD.pdf.

[12] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (TranslationManager),” Eclipse Arrowhead, IDD 5.3.0, 2026, translationManager-v5.3.0-generalManagement-MQTT-IDD.pdf.

[13] ——, “sysMonitor – Service Description (TranslationManager),” Eclipse Arrowhead, SD 5.3.0, 2026, translationManager-v5.3.0-sysMonitor-SD.pdf.

[14] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (TranslationManager),” Eclipse Arrowhead, IDD 5.3.0, 2026, translationManager-v5.3.0-sysMonitor-AMQP-IDD.pdf.

6 Revision History

6.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)
3	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

6.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	